

Self-Balancing Upside Down Pendulum

By tmoir in Circuits > Robots  25,353  40  34



- This is my attempt at the inverted pendulum balancing on a two-wheel chassis cart. I had seen the other ones on here and was inspired to try my own version. The Arduino is wonderfully simple to program and you can pick it up in a few hours. I looked at the Segways as well and compared their approach - which is quite similar. Unlike most of the others I don't use PID control, but State-Feedback control. My background is in control-systems so I found it very interesting. I feedback angular position, velocity and acceleration and add integral action. The integral action is needed to keep the steady-state error at zero. Without it the pendulum flops back and forth and doesn't stay as straight. The pendulum itself is just a length of aluminum (about 500mm long) with an aluminum bob screwed on. I didn't make this part myself but had it lying around from a previous inverted pendulum project on an old x-y plotter which in itself was quite

interesting. You could use a piece of bamboo instead of whatever you have lying around. It turns out that the longer is the pendulum shaft the easier it is to control. Try balancing a pencil on your hand - now try a long broom handle and see how much easier it is. You may think that you are wasting your time experimenting with such things, but remember that the control system that keeps a rocket in the air or a walking robot all works on similar principles (but much harder!) Of course the Segway also works along similar lines except you are standing on it and you become part of the control loop. Here is a video of some Auckland University of Technology (AUT) students with the original pendulum (a larger one) on an x-y plotter. entirely analogue. This inspired me to do the one on wheels during the Xmas holidays! Note the structural resonance of the shaft. All mechanical systems have structural resonances and they are a headache when putting feedback around them when operating at high bandwidth. You normally require as high a bandwidth as possible for speed of response.

Step 1: Gather Parts



You will need the dc motors chassis.2 car chassis DC geared motor drive 150 rpm 6V / 12V optional wheels 65mm. Obtainable from

taobao.com

<http://item.taobao.com/item.htm?spm=a1z10.5-c.w400...>

or in English from aliexpress

<http://www.aliexpress.com/store/product/2-wheel-dr...>

They are quality wheels though not cheap.You can of course use cheaper ones. I prefer the expensive ones since I will be doing a lot of rough work on them!

You will also need a L298 H bridge - see this instructable

<https://www.instructables.com/id/Arduino-Modules-L2...>

Buy here

<https://www.sparkfun.com/products/9670>

You will also need an Arduino Uno

<http://arduino.cc/en/main/arduinoBoardUno>

You will also need connecting wires for the Arduino so you don't have to solder - they just plug into the pins. Female at both ends. Buy from Sparkfun or Hobbytronics

<http://www.hobbytronics.co.uk/cables-connectors>

To get the male ending for the Arduino pin socket I just used header pins I had lying about. Break them up as required and bend them into right angles.

<http://www.hobbytronics.co.uk/cables-connectors/pc...>

1 LiPo battery 11.1v will do - because they are powerful and lightweight. Suggest you buy a custom charger with it. Best to be safe rather than set fire to your house! Can buy a decent battery + charger from HobbyKing

http://www.hobbyking.com/hobbyking/store/_8934_T...

http://www.hobbyking.com/hobbyking/store/_43038_...

I have over-engineered this a bit but at least the batteries last a long time between charges.

Finally you will need the Gyro-accelerometer - possibly the cheapest of all the parts, the MPU6050. I bought about 4 just in case for other projects. Just search for it on Bay. About \$2 US each for the breakout board! You will need to solder header pins onto it. It runs off 3.3V which your Arduino Uno has as an output.

Misc things you will need are a few 10k pots and a battery plug for your Arduino. You will also need a sheet of suitably cut plexiglass or a piece of plastic. I live in New-Zealand and we can get this from Jaycar

<http://www.jaycar.com.au/productView.asp?ID=HM9509>

You can also buy larger sheets from hardware stores.

I strongly suggest buying zip-ties to bundle the cables together so that the whole thing is robust. It will fall over many times during tests so it needs to be sturdy.

A toggle switch is needed to power on and off the unit.

Step 2: The Chassis



As you can see from the picture, the chassis has the plexiglass sheet bolted onto it. To do this you need to bend the plexiglass at right angles. Fortunately I have a hot-wire system for doing this at work but it is possible to use a heat gun as used for stripping paint off walls! You need to be quick and practice though, otherwise you'll get the wrong shape. Then just drill some holes and attach to the motor chassis. If you don't want to use this approach then you can stack everything on top of each other as others have done. Whatever you do, keep the Gyro as close to the middle of the axle as possible and lay it flat with the pins sticking up. don't mount the Gyro half way up or you will get an error with your readings. I didn't put the aluminum pendulum rod on at first - it is just held on by drilling a few holes and claspings it with cable-ties (zip-ties). Instead of the plexiglass sheet you could use a plastic box maybe if you can find one the right width. Here is an instructional video on how to bend plexiglass.

How to : Bending & Shaping Plastic with hot air gun, heat gun

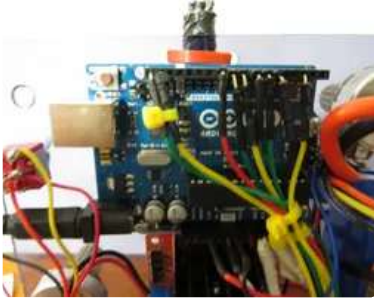
Instructional Videos-Steinel Heat Guns & Glue Guns



Watch on

Once the chassis is built don't be tempted to just wire everything up and try the complete system. you first need to test out the motor control via the H bridge on its own - no Gyro (using the Arduino) and then test the Gyro with the Arduino observing the tilt angles in real-time - and no motors. The wiring will be the same to the Arduino when we do the complete pendulum so no need to re-wire.

Step 3: Controlling the Dc Motors Using PWM - No Gyro Connections



So you initially need to mount the Uno and the L298 H bridge. Just drill some holes and attach via small screws and spacers if you like. Because it's a plastic backing you need not worry about insulation. I mounted the Uno at the top and the H bridge below it. The battery is held by zip-ties. Originally I used an external power supply to do the initial testing. I suggest you wire up the L298 Bridge to the dc motors and do some testing first. The motors are driven by PWM waveforms. The waveforms which come out of the Arduino are shown in the diagram above. The Arduino cannot drive a motor directly so this PWM drives an H-Bridge controller to step up the voltage and increase the driving current. Some H bridges have two logic inputs - one is held High and the other Low for forward, and the opposite for reverse. This is the case for the one we use here. Others have internal logic and just use High/Low for forward and reverse. The `analogWrite(pin,number from 0 to 255)` command gives the strength of the PWM which essentially when averaged out gives us a dc component which varies from 0 to a maximum 255 for full speed!! For example with `analogWrite(10,255)` writes max PWM to pin 10 (which is just dc 5v) , `analogWrite(10,128)` gives 50% speed to pin 10 and `analogWrite(10,64)` gives quarter speed. Pin 10 would then have to be connected to the PWM input of your H bridge. See waveform diagrams.

Read this first

<https://www.instructables.com/id/Arduino-Modules-L2...>

We will use the 5V regulated voltage output for the potentiometers later.

Then follow the tutorial on dc motors by John Boxall

<http://tronixlabs.com/news/tutorial-l298n-dual-mot...>

follow his connections as we will use them in our program.

Then run his example and check that both motors turn in the same direction!! If not then reverse the polarity on one of the offending motors. I leaned a lot from his Arduino tutorial.

Connections are as follows:

Arduino-----> L298 H Bridge

D10 -----> 7

D5 ----->12

D9-----> IN1

D8-----> IN2

D7 -----> IN3

D6-----> IN4

Plus the motor connections.

Don't forget to connect zero volts from the Arduino to the zero volts on the H-Bridge. In fact the zero volts connector on the H bridge board became rather crowded and I ended up soldering all of them together to make them robust.

His demo program is as follows. Make sure it works first before continuing to the Gyro part.

```
// connect motor controller pins/to Arduino digital pins
```

```
// motor one
```

```
int enA = 10;
```

```
int in1 = 9;
```

```
int in2 = 8;
```

```
// motor two
```

```
int enB = 5;
```

```
int in3 = 7;
```

```
int in4 = 6;
```

```
void setup()
```

```
{
```

```
// set all the  
motor control pins to outputs
```

```
pinMode(enA,OUTPUT);
```

```
pinMode(enB,OUTPUT);
```

```
pinMode(in1,OUTPUT);
```

```
pinMode(in2,OUTPUT);
```

```
pinMode(in3, OUTPUT);

pinMode(in4,OUTPUT);

}

void demoOne()

{

// this function will run the motors in both directions at a fixed speed

// turn on motor A

digitalWrite(in1,HIGH);

digitalWrite(in2,,LOW);

// set speed to 200 out of possible range 0~255

analogWrite(enA,200);

// turn on motor B

digitalWrite(in3,HIGH);

digitalWrite(in4,LOW);

// set speed to 200 out of possible range 0~255

analogWrite(enB,200);

delay(2000);

// now change motor directions

digitalWrite(in1,LOW);

digitalWrite(in2,HIGH);

digitalWrite(in3,LOW);

digitalWrite(in4,HIGH);

delay(2000);

// now turn off motors

digitalWrite(in1,LOW);

digitalWrite(in2,LOW);

digitalWrite(in3,LOW);

digitalWrite(in4,LOW);
```

```
}

void demoTwo()

{

// this function will run the motors across the range of possible speeds

// note that maximum speed is determined by the motor itself and the operating voltage

// the PWM values sent by analogWrite() are fractions of the maximum speed possible by

// turn on motors

digitalWrite(in1,LOW);

digitalWrite(in2,HIGH);

digitalWrite(in3,LOW);

digitalWrite(in4,HIGH);

// accelerate from zero to maximum speed

for (int i = 0; i< 256; i++)

{

analogWrite(enA,i);

analogWrite(enB,i);

delay(20);

}

// decelerate from maximum speed to zero

for (int i = 255; i>= 0; --i)

{

analogWrite(enA,i);

analogWrite(enB,i);

delay(20);

}

// now turn off motors

digitalWrite(in1,LOW);

digitalWrite(in2,LOW);
```

```
digitalWrite(in3,LOW);

digitalWrite(in4,LOW);

}

void loop()

{

demoOne();

delay(1000);

demoTwo();

delay(1000);

}
```

Step 4: The MPU6050 3 Axis Gyro and Accelerometer



What we need is a signal proportional to the angle of tilt - which is the Y axis in the software and this Gyro when it is sitting flat. Once again I don't recommend just jumping to the final solution I have, test the Gyro on its own first and watch on the serial port what the angle is as you tilt it back and forth (with no motor drive). so you need to use the program from this link by Jeff Rowberg who wrote the driver software

<https://github.com/jrowberg/i2cdevlib/blob/master/...>

You will also need to use his library (same link). If his program does not compile then you haven't got his library

i2cdevlib installed on the Arduino. You can download it from this indestructible

<https://www.instructables.com/id/Balancing-Instruct...>

When you run the test program you may well see that when the pendulum is at 90 degrees vertical that the angle is offset by say anything up to +/- 10 degrees!! This varies from

device to device (I am led to believe) and that is why I put a trim pot on the final design.

The wiring of the MPU6050 Gyro is straightforward

Connect VCC and Gnd to 3.3v and ground of the Arduino (NOT 5V!)

connect Int (the interrupt) pin to Arduino Pin 2

connect SCL and XDA to the corresponding same lettered pins on the Arduino. This is for the i2c bus.

The other pins are not used.

I mounted the Gyro on a small piece of plastic via two small screws (holes are already drilled on the device of course). I then used a glue-gun to glue the assembly to the wheel chassis. Here is the test program for the Gyro. It doesn't power the motors, just use it to test. Use the serial monitor on the Arduino to see Gyro outputs.

// I2C device class (I2Cdev) demonstration Arduino sketch for MPU6050 class using DMP
(MotionApps v2.0)

// 6/21/2012 by Jeff Rowberg

// Updates should (hopefully) always be available at <https://github.com/jrowberg/i2cdevlib>

Changelog:

// 2013-05-08 - added seamless Fastwire support

// - added note about gyro calibration

// 2012-06-21 - added note about Arduino 1.0.1 + Leonardo compatibility error

// 2012-06-20 - improved FIFO overflow handling and simplified read process

// 2012-06-19 - completely rearranged DMP initialization code and simplification

// 2012-06-13 - pull gyro and accel data from FIFO packet instead of reading directly

// 2012-06-09 - fix broken FIFO read sequence and change interrupt detection to RISING

// 2012-06-05 - add gravity-compensated initial reference frame acceleration output

// - add 3D math helper file to DMP6 example sketch

// - add Euler output and Yaw/Pitch/Roll output formats

// 2012-06-04 - remove accel offset clearing for better results (thanks Sungon Lee)

// 2012-06-01 - fixed gyro sensitivity to be 2000 deg/sec instead of 250

// 2012-05-30 - basic DMP initialization working

```
/* =====
```

I2Cdev device library code is placed under the MIT license

Copyright (c) 2012 Jeff Rowberg

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR

IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,

FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER

LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,

OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN

THE SOFTWARE.

```
=====
```

```
*
```

```
/ I2Cdev and MPU6050 must be installed as libraries, or else the .cpp/.h files
```

```
// for both classes must be in the include path of your project
```

```
#include "I2Cdev.h"
```

```
#include "MPU6050_6Axis_MotionApps20.h"
```

```
//#include "MPU6050.h" // not necessary if using MotionApps include file

// Arduino Wire library is required if I2Cdev I2CDEV_ARDUINO_WIRE implementation
// is used in I2Cdev.h

#if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE

#include "Wire.h"

#endif

// class default I2C address is 0x68

// specific I2C addresses may be passed as a parameter here

// AD0 low = 0x68 (default for SparkFun breakout and InvenSense evaluation board)

// AD0 high = 0x69

MPU6050 mpu;

//MPU6050 mpu(0x69); // <-- use for AD0 high
```

```
/*
=====
=====
```

NOTE: In addition to connection 3.3v, GND, SDA, and SCL, this sketch depends on the MPU-6050's INT pin being connected to the Arduino's external interrupt #0 pin. On the Arduino Uno and Mega 2560, this is digital I/O pin 2.

```
*
=====
===== *

*
=====
=====
```

NOTE: Arduino v1.0.1 with the Leonardo board generates a compile error when using `Serial.write(buf, len)`. The Teapot output uses this method. The solution requires a modification to the Arduino USBAPI.h file, which is fortunately simple, but annoying. This will be fixed in the next IDE release. For more info, see these links:

<http://arduino.cc/forum/index.php/topic,109987.0.h...>

<http://code.google.com/p/arduino/issues/detail?id=...>

*

=====

===== */

// uncomment "OUTPUT_READABLE_QUATERNION" if you want to see the actual

// quaternion components in a [w, x, y, z] format (not best for parsing

// on a remote host such as Processing or something though)

//#define OUTPUT_READABLE_QUATERNION

// uncomment "OUTPUT_READABLE_EULER" if you want to see Euler angles

// (in degrees) calculated from the quaternions coming from the FIFO.

// Note that Euler angles suffer from gimbal lock (for more info, see

// http://en.wikipedia.org/wiki/Gimbal_lock)

//#define OUTPUT_READABLE_EULER

// uncomment "OUTPUT_READABLE_YAWPITCHROLL" if you want to see the yaw

/ pitch/roll angles (in degrees) calculated from the quaternions coming

// from the FIFO. Note this also requires gravity vector calculations.

// Also note that yaw/pitch/roll angles suffer from gimbal lock (for

// more info, see: http://en.wikipedia.org/wiki/Gimbal_lock)

#define OUTPUT_READABLE_YAWPITCHROLL

// uncomment "OUTPUT_READABLE_REALACCEL" if you want to see acceleration

// components with gravity removed. This acceleration reference frame is

// not compensated for orientation, so +X is always +X according to the

// sensor, just without the effects of gravity. If you want acceleration

// compensated for orientation, us OUTPUT_READABLE_WORLDACCEL instead.

//#define OUTPUT_READABLE_REALACCEL

// uncomment "OUTPUT_READABLE_WORLDACCEL" if you want to see acceleration

// components with gravity removed and adjusted for the world frame of

// reference (yaw is relative to initial orientation, since no magnetometer

// is present in this case). Could be quite handy in some cases.

```
//#define OUTPUT_READABLE_WORLDACCEL
```

```
// uncomment "OUTPUT_TEAPOT" if you want output that matches the
```

```
// format used for the InvenSense teapot demo
```

```
//#define OUTPUT_TEAPOT
```

```
#define LED_PIN 13 // (Arduino is 13, Teensy is 11, Teensy++ is 6)
```

```
bool blinkState = false;
```

```
// MPU control/status vars
```

```
bool dmpReady = false; // set true if DMP init was successful
```

```
uint8_t mpulntStatus; // holds actual interrupt status byte from MPU
```

```
uint8_t devStatus; // return status after each device operation (0 = success, !0 = error)
```

```
uint16_t packetSize; // expected DMP packet size (default is 42 bytes)
```

```
uint16_t fifoCount; // count of all bytes currently in FIFO
```

```
uint8_t fifoBuffer[64]; // FIFO storage buffer
```

```
// orientation/motion vars
```

```
Quaternion q; // [w, x, y, z] quaternion container
```

```
VectorInt16 aa; // [x, y, z] accel sensor measurements
```

```
VectorInt16 aaReal; // [x, y, z] gravity-free accel sensor measurements
```

```
VectorInt16 aaWorld; // [x, y, z] world-frame accel sensor measurements
```

```
VectorFloat gravity; // [x, y, z] gravity vector
```

```
float euler[3]; // [psi, theta, phi] Euler angle container
```

```
float ypr[3]; // [yaw, pitch, roll] yaw/pitch/roll container and gravity vector
```

```
// packet structure for InvenSense teapot demo
```

```
uint8_t teapotPacket[14] = { '$', 0x02, 0,0, 0,0, 0,0, 0x00, 0x00, '\r', '\n' };
```

```
// =====
```

```
// === INTERRUPT DETECTION ROUTINE ===
```

```
// =====
```

```
volatile bool mpulInterrupt = false; // indicates whether MPU interrupt pin has gone high
```

```
void dmpDataReady() {

mpuInterrupt = true;

}

// =====

// === INITIAL SETUP ===

// =====

void setup() {

// join I2C bus (I2Cdev library doesn't do this automatically)

#if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE

Wire.begin();

TWBR = 24; // 400kHz I2C clock (200kHz if CPU is 8MHz)

#elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE

Fastwire::setup(400, true);

#endif

// initialize serial communication

// (115200 chosen because it is required for Teapot Demo output, but it's

// really up to you depending on your project)

Serial.begin(115200);

while (!Serial); // wait for Leonardo enumeration, others continue immediately

// NOTE: 8MHz or slower host processors, like the Teensy @ 3.3v or Arduinio

// Pro Mini running at 3.3v, cannot handle this baud rate reliably due to

// the baud timing being too misaligned with processor ticks. You must use

// 38400 or slower in these cases, or use some kind of external separate

// crystal solution for the UART timer.

// initialize device

Serial.println(F("Initializing I2C devices..."));

mpu.initialize();

// verify connection
```

```
Serial.println(F("Testing device connections..."));

Serial.println(mpu.testConnection() ? F("MPU6050 connection successful") : F("MPU6050
connection failed"));

// wait for ready

Serial.println(F("\nSend any character to begin DMP programming and demo: "));

while (Serial.available() && Serial.read()); // empty buffer

while (!Serial.available()); // wait for data

while (Serial.available() && Serial.read()); // empty buffer again

// load and configure the DMP

Serial.println(F("Initializing DMP..."));

devStatus = mpu.dmpInitialize();

// supply your own gyro offsets here, scaled for min sensitivity

mpu.setXGyroOffset(220);

mpu.setYGyroOffset(76);

mpu.setZGyroOffset(-85);

mpu.setZAccelOffset(1788); // 1688 factory default for my test chip

// make sure it worked (returns 0 if so)

if (devStatus == 0) {

// turn on the DMP, now that it's ready

Serial.println(F("Enabling DMP..."));

mpu.setDMPEnabled(true);

// enable Arduino interrupt detection

Serial.println(F("Enabling interrupt detection (Arduino external interrupt 0)..."));

attachInterrupt(0, dmpDataReady, RISING);

mpuIntStatus = mpu.getIntStatus();

// set our DMP Ready flag so the main loop() function knows it's okay to use it

Serial.println(F("DMP ready! Waiting for first interrupt..."));

dmpReady = true;
```

```
// get expected DMP packet size for later comparison

packetSize = mpu.dmpGetFIFOPacketSize();

} else {

// ERROR!

// 1 = initial memory load failed

// 2 = DMP configuration updates failed

// (if it's going to break, usually the code will be 1)

Serial.print(F("DMP Initialization failed (code ");

Serial.print(devStatus);

Serial.println(F(""));

}

// configure LED for output

pinMode(LED_PIN, OUTPUT);

}

// =====

// === MAIN PROGRAM LOOP ===

// =====

void loop() {

// if programming failed, don't try to do anything

if (!dmpReady) return;

// wait for MPU interrupt or extra packet(s) available

while (!mpuInterrupt && fifoCount < packetSize) {

// other program behavior stuff here

// .

// .

// .

// if you are really paranoid you can frequently test in between other

// stuff to see if mpuInterrupt is true, and if so, "break;" from the
```

```
// while() loop to immediately process the MPU data

// .

// .

// .

}

// reset interrupt flag and get INT_STATUS byte

mpuInterrupt = false;

mpuIntStatus = mpu.getIntStatus();

// get current FIFO count

fifoCount = mpu.getFIFOCount();

// check for overflow (this should never happen unless our code is too inefficient)

if ((mpuIntStatus & 0x10) || fifoCount == 1024) {

// reset so we can continue cleanly

mpu.resetFIFO();

Serial.println(F("FIFO overflow!"));

// otherwise, check for DMP data ready interrupt (this should happen frequently)

} else if (mpuIntStatus & 0x02) {

// wait for correct available data length, should be a VERY short wait

while (fifoCount < packetSize) fifoCount = mpu.getFIFOCount();

// read a packet from FIFO

mpu.getFIFOBytes(fifoBuffer, packetSize);

// track FIFO count here in case there is > 1 packet available

// (this lets us immediately read more without waiting for an interrupt)

fifoCount -= packetSize;

#ifdef OUTPUT_READABLE_QUATERNION

// display quaternion values in easy matrix form: w x y z

mpu.dmpGetQuaternion(&q, fifoBuffer);

Serial.print("quat\t");
```

```
Serial.print(q.w);

Serial.print("\t");

Serial.print(q.x);

Serial.print("\t");

Serial.print(q.y);

Serial.print("\t");

Serial.println(q.z);

#endif

#ifdef OUTPUT_READABLE_EULER

// display Euler angles in degrees

mpu.dmpGetQuaternion(&q, fifoBuffer);

mpu.dmpGetEuler(euler, &q);

Serial.print("euler\t");

Serial.print(euler[0] * 180/M_PI);

Serial.print("\t");

Serial.print(euler[1] * 180/M_PI);

Serial.print("\t");

Serial.println(euler[2] * 180/M_PI);

#endif

#ifdef OUTPUT_READABLE_YAWPITCHROLL

// display Euler angles in degrees

mpu.dmpGetQuaternion(&q, fifoBuffer);

mpu.dmpGetGravity(&gravity, &q);

mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

Serial.print("ypr\t")

Serial.print(ypr[0] * 180/M_PI);

Serial.print("\t");

////////////////////////////////////////////////////////////////
```

/ This is the Gyro angle ypr[1], we use this one for the pendulum cart and convert to degrees in the line below.

// make sure you get close to zero when the cart is vertical. b n

```
Serial.print(ypr[1] * 180/M_PI);
```

```
Serial.print("\t");
```

```
////////////////////////////////////
```

```
Serial.println(ypr[2] * 180/M_PI);
```

```
#endif
```

```
#ifdef OUTPUT_READABLE_REALACCEL
```

```
// display real acceleration, adjusted to remove gravity
```

```
mpu.dmpGetQuaternion(&q, fifoBuffer);
```

```
mpu.dmpGetAccel(&aa, fifoBuffer);
```

```
mpu.dmpGetGravity(&gravity, &q);
```

```
mpu.dmpGetLinearAccel(&aaReal, &aa, &gravity);
```

```
Serial.print("areal\t");
```

```
Serial.print(aaReal.x);
```

```
Serial.print("\t");
```

```
Serial.print(aaReal.y);
```

```
Serial.print("\t");
```

```
Serial.println(aaReal.z);
```

```
#endif
```

```
#ifdef OUTPUT_READABLE_WORLDACCEL
```

```
// display initial world-frame acceleration, adjusted to remove gravity
```

```
// and rotated based on known orientation from quaternion
```

```
mpu.dmpGetQuaternion(&q, fifoBuffer);
```

```
mpu.dmpGetAccel(&aa, fifoBuffer);
```

```
mpu.dmpGetGravity(&gravity, &q);
```

```
mpu.dmpGetLinearAccel(&aaReal, &aa, &gravity);
```

```
mpu.dmpGetLinearAccelInWorld(&aaWorld, &aaReal, &q);

Serial.print("aworld\t");

Serial.print(aaWorld.x);

Serial.print("\t");

Serial.print(aaWorld.y);

Serial.print("\t");

Serial.println(aaWorld.z);

#endif

#ifdef OUTPUT_TEAPOT

// display quaternion values in InvenSense Teapot demo format:

teapotPacket[2] = fifoBuffer[0];

teapotPacket[3] = fifoBuffer[1];

teapotPacket[4] = fifoBuffer[4];

teapotPacket[5] = fifoBuffer[5];

teapotPacket[6] = fifoBuffer[8];

teapotPacket[7] = fifoBuffer[9];

teapotPacket[8] = fifoBuffer[12];

teapotPacket[9] = fifoBuffer[13];

Serial.write(teapotPacket, 14);

teapotPacket[11]++; // packetCount, loops at 0xFF on purpose

#endif

// blink LED to indicate activity

blinkState = !blinkState;

digitalWrite(LED_PIN, blinkState);

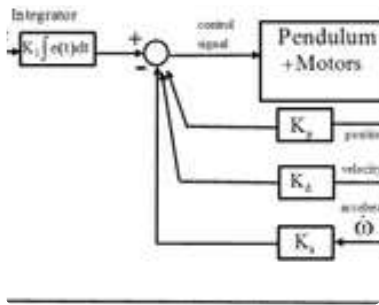
}

}
```

Step 5: The Final System

Once you have tested the motors using the test program and the Gyro using a second test program as per the previous steps then you are nearly ready for the final software. I added two 10K potentiometers - both connected across the regulated 5v to zero volt line (from the H bridge). Then I connected the wipers of the pots to A0 and A1 of the analogue to digital convertor of the Arduino. In this way I can read in dc voltages and create a small offset via A1 and an overall gain from A0. Other than that the wiring is the same as per the two test programs. The overall software is shown in the attached text file.

Step 6: The Hard Sums!



So how does it balance? Negative feedback is used by creating an error between the desired angle of inclination and the measured value from the gyro. If the two differ then a control voltage is applied to both motors simultaneously to drive the cart forward or backwards to correct this angle. Such an approach is known as Proportional control and suffers from the fact that this can be quite a large error that is created. Theory tells us that by increasing the amount of gain to the error path that this error reduces. Unfortunately to make the error small enough requires such a large gain that the loop becomes unstable and the cart falls over. We can improve matters by adding a second term which is proportional to the differential of the error signal time a second gain term. This is known as Differential control and when added to the Proportional term is known as PD or Proportional plus Derivative control. The D term stabilizes the system and reduces the instability. That would be the answer except that the error doesn't always go to zero and there is a finite difference left over. The cart will flop back and forward and not stay up straight. We can add a third term which is an integrator which takes the error away. Unfortunately this third so-called I term if given in too big a quantity leads to further instability. Hence a compromise must be met between the three terms in what is known as a PID controller. Tuning PID controllers is a bit of an art and there are tuning rules, but they aren't quite so good when you have an inherently unstable system as we do. However, PID does work and other instructibles have used it.

The approach we use here is the approach that many are using for the Segway and is not PID control. PID is often known as Classical control theory whereas what we use here is known as Modern Control theory. In fact the Classical approach isn't that old and the Modern approach isn't that modern! The Modern approach we use is known as Pole-Placement State-Feedback. In it we feedback proportions of angular position, velocity and acceleration. The Segway indestructibles only use position and velocity feedback to the best of my knowledge. I added a third state of acceleration and a fourth state which is integration or integral action. This sort of thing is well established in text books in fact but seldom seen in demonstrations. We would normally require a mathematical model i.e the differential equations describing the complete dynamic system. Here I use a simpler approach and tune it like a PID. There is a similarity between PD control and state feedback that only uses position and velocity feedback. The velocity feedback is known as

Rate feedback and is just a differential term which is read from a sensor rather than being explicitly calculated. The difference here is that differentiation is in the Feedback path rather than the forward path as is the case with PID. This gives a little more damping at the expense of a slightly slower system - but can easily be compensated for by increasing the overall gain. So here we measure directly angular error using the Gyro and angular velocity from the accelerometer.

The third state is acceleration which we compute (there are better ways but for now I use this method) directly by differentiating velocity. By controlling acceleration we can control the current drawn from the motors and improve disturbance rejection. If you cannot measure acceleration with a transducer (which is the usual case) then you must create it by differentiation or by using a state Observer or Kalman filter.

So the control signal is

$$u(k) = -K[K_p y(k) + K_d y(k)_{\dot{}} + K_a y(k)_{\ddot{}}]$$

where K is overall gain adjusted from the potentiometer and K_p , K_d and K_a are the proportional gain, derivative gain and acceleration gain respectively. Here $y(k)$ represents the present reading from the Gyro sensor (angle), $y(k)_{\dot{}}$ is present reading of angular velocity and $y(k)_{\ddot{}}$ is the present reading of acceleration all at time instant k. In the diagram K_i is integral gain.

To digitally differentiate we can use a simple Euler approach. For a sampling interval dT seconds (here it is $dT=0.01$ for a 100Hz sampling rate) we can differentiate velocity ω to give acceleration accel as

```
// calculate angular acceleration from angular velocity
```

```
omega_old = omega; // store last value of acceleration
```

```
omega = ...read velocity from accelerometer here
```

```
// differentiate
```

```
accel=(omega-omega_old)/dT;
```

Usually we leave dT out (i.e set $dT=1$) as it becomes a gain term which we can adjust separately.

This is quite a crude differentiator and in fact any type of pure differentiator is a bad idea for a control loop because it amplifies high-frequency noise and excites structural resonance. Therefore we low-pass filter the differentiated output. we use a first order low-pass filter - would take a while to explain all of the theory here. this filter is not shown on the diagram above. (only the ideal case).

To integrate we use a simple Euler integration. eg to integrate an error

```
//Integration of error
```

```
// setpoint = 0 for zero degrees vertical
```

```
error=setpoint-output_value;
```

```
y_out_past=y_out;/store /past value of integrator output
```

```
y-out=y_out_past+dT*error;
```

```
// yes you can use the y+= type notation in code but it hides what is happening and the understanding
```

where `y_out` is the integrated output. usually we leave out `dT` and scale later i.e `dT=1`. You could in principle use a summation to do integration, but the above method is a better - recursive method sample by sample and only requires minimal storage. Summation would require us to store a number of past values. You can also perform integration via z-transforms and get a slightly more accurate version called a Trapezoidal Integrator. No need for that complication here, though I did use z-transforms for our simple low-pass filter.

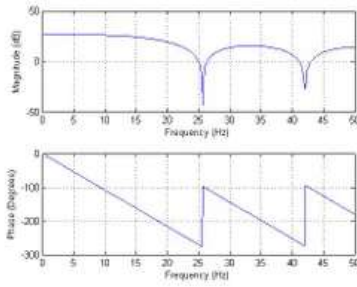
The sampling frequency: I used previous Segway code to set the sampling frequency to 100Hz and could not get it to go faster. I checked with an oscilloscope that this was the true sampling frequency by flipping a logic flag up and down - true and false very execution of the loop. I send this to a digital out and got a square wave. I measured the half-period and it was spot on 0.01secs=`dT`. The block diagram is shown with full state-feedback and integral action.

I also used the special so-called complementary filters to combine the accelerometer and gyro readings. This is because the accelerometer is good at low frequencies whilst the gyro is good at high frequencies whilst the opposite is the case for poor performance. This attempts to reduce the effect of gyro drift. Another approach is to use a Kalman filter but this is not shown here. Details of complementary filter method is shown in the link below.

http://www.chrismarion.net/index.php?option=com_co...

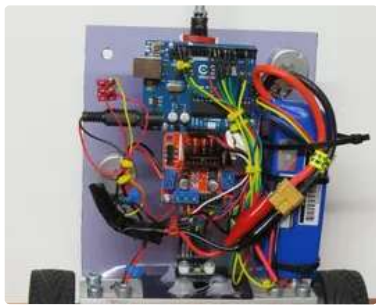
The main idea is shown in the diagram (shown below the state-feedback diagram) taken from this link.

Step 7: A Note on the Savitsky Golay Filter



Many Segway projects appear to use what is known as a Savitsky Golay filter to try and average out the Gyro readings and remove erroneous readings. This special filter is what is known as a finite-impulse filter (FIR) and unfortunately its phase-shift is not too good. Any negative phase-shift has a destabilizing effect on a control loop. so when you do ordinary filtering of say music or speech, phase maybe isn't as important, but when you are working inside a control-loop you need to be careful and not be wasteful on phase-lag. Whilst I agree that we do need a filter, I do not agree that this filter is suitable. I tried it and it caused less stability than without it. You could trade this off by going to an even lower bandwidth I suppose, but I decided to just use a simple low-pass filter instead. $(1/(1+s\tau))$ if you understand such things. Make sure the cut-off was far enough away (14Hz) from the unity-gain crossover frequency of the loop (which I take to be around 1 Hz). A rule often used in control-engineering is to sample at least 10 times higher than the highest frequency of interest. The usual Nyquist sampling rate in signal processing is taken to be only twice as high but in control it needs to be much higher because digital control has an inherent one-step time-delay from input to output due to the computer! Any delay in a control loop causes negative going phase which in turn destabilizes the loop. The Savitsky Golay filter frequency and phase response (top graph) was plotted on MATLAB and although it has a nice magnitude response it's phase turns hard negative with a steep slope. In contrast the simple first-order filter I use (bottom graph) has a minor phase shift only at low frequencies. The main purpose of my filter is to attenuate structural resonances at higher frequencies. Without it the whole mechanism shakes! Another problem with some FIR filters is that they can quite often be non-minimum phase - and this one is just that. That means it gives much greater phase shift than an equivalent filter that is minimum phase.

Step 8: Switching On



It's best to tun the gain down to zero on your gain potentiometer and monitor the phase angle coming from the Gyro. Then adjust the offset pot so that the output as viewed on the serial port is close to zero when vertical.

You can then increase the gain slowly until it balances. I have a video of it working

Inverted Pendulum using State-Feedback.

T.J Moir



Watch on

You may of course have to change the various state-feedback gains for your system. You can do this fastest by having pots for each gain - which is a lot of pots of course! I did it the hard way in software. Now a demonstration of what I mean by structural resonance. I deliberately set the gain too high and the entire structure shakes.

Instability and Structural Resonance Illustrated for a Servo

T.J Moir



Watch on

I also have a demo of the PID version. Similar to other projects here but this one has a filter on the derivative action.

Inverted pendulum cart

T.J Moir



Watch on

I then swapped the smaller half metre pendulum with the one metre one. I had to change the parameters of the program. I reduced the filter 3dB frequencies to 7Hz (from 14Hz). This attenuated structural resonances. I also had to tweak a few more of the state-feedback gains and add a little more integral action on the error.

One metre inverted pendulum cart

T.J Moir



Watch on

The arduino file for the larger pendulum is attached as BFP1_inf

Step 9: Epilogue - an Improvement.



- I found that although the system worked, it didn't stay balanced for nearly long enough - which it should have done. The solution was to use different motors with a gear reduction of around 30:1
- <http://www.aliexpress.com/store/product/Two-self-b...>
- These motors also have encoders built in which will be useful for future modifications.
- The system then worked nearly perfectly and was very robust. You can knock it and it still stays up and even balance things on the top as illustrated in the video below. The original perspex chassis for the project broke when I dropped it on the ground and so the one you see in the video is a smarter one knocked up in a few hours by one of our best technicians using a laser cutter.

State-Space control of an Inverted pendulum cart. State-feedback plus integr

T.J Moir



Watch on

I found that all of the electronic circuit boards lifted off all at once linked together by the wiring and I managed to slot them into an exact replica made from the original template-only better looking. The job took about 15 mins.